# Pasteur Network Climate and Health Metadata Catalogue - how it fits together

One source, two ways of reading it.

| File | What it is | Who uses it |
|---|---|---|
| `catalogue.json` | The source. One object per dataset, one key per field. | You, when adding or editing a record |
| `climate_health_catalogue_register.csv` | The same content as a flat table. Opens in Excel or Google Sheets; import into CKAN or Airtable. | Partner agencies filling in their own rows |
| `climate_health_catalogue.html` | Single-file web page. Search, filter by project, domain and access, expand any record. No server and no build step. Nothing is fetched over the network - the typefaces are system fonts and the logo is embedded - so it works offline and still looks right when emailed. | Anyone who needs to look something up |
| `climate_health_catalogue.docx` | Printable and editable. One page per dataset, a summary table, the open gaps, the field definitions and a change log. **Not in the repository** - it embeds the gaps register, so it is built locally and shared directly. | The revewer report, and anyone who prefers paper |
| `gaps.json` | The open gaps register - internal review material. **Not in the repository.** Kept beside the checkout; `build_docx.js` reads it if present. | The laboratory, when reviewing progress |

## The loop

Edit `catalogue.json`, then rebuild both outputs:

```
python3 build_web.py     # validates catalogue.json, writes the CSV and the HTML
node   build_docx.js     # writes the DOCX   (first run: npm install)
```

`build_web.py` needs nothing beyond the standard library. `build_docx.js` needs the `docx` package declared in `package.json`, so a fresh clone runs `npm install` once.

The two PDFs are built from their Markdown by `convert2pdf.py`, which lives here:

```
python3 convert2pdf.py README.md README.pdf
python3 convert2pdf.py EDITING_GUIDE.md EDITING_GUIDE.pdf
```

Its dependencies, and those of its sibling `convert2docx.py`, are in `requirements.txt` (`pip install -r requirements.txt`). Both converters are general Markdown tools and are not involved in building the catalogue itself.

`build_docx.js` also reads `gaps.json` if it is beside the checkout. Without it the DOCX builds without the gaps section - which is what anyone cloning the public repository gets, and is intended.

If you would rather work in a spreadsheet, edit the CSV and convert it back to JSON before rebuilding. Do not edit the DOCX and expect the change to survive a rebuild - copy it back into the source first.

## Publishing the web page

The HTML is one file, so it travels anywhere - put it on a Pasteur Network web server, or email it and it opens from disk. It has no external references at all: the typefaces are the ones the Network's own site uses, which resolve from system fonts, and the logo is embedded in the file as a data URI. Embedding the logo is why the page is 196,215 bytes rather than the 122,825 it was, and it is the reason the page renders identically with no network at all.

It is also published as a GitHub Page:

**https://pasteurnetwork.github.io/climate-health-data-catalogue/**

To update it, rebuild and push:

```
python3 build_web.py
git commit -am "Update catalogue"
git push
```

GitHub rebuilds on every push to `main`. The new version is live in under a minute.

Three things about that setup:

- **Only `docs/` is published as a page.** Pages serves that folder and nothing above it. `build_web.py` writes the page into `docs/` and the register into the repo root - that split is the whole mechanism, so do not move files between the two without meaning it.
- `docs/index.html` is a six-line redirect to `climate_health_catalogue.html`. GitHub Pages serves `index.html` at the site root, and without it the bare URL 404s. It holds no copy of the page, so it never goes stale.
- **Treat everything committed as public.** The repository is private today but is prepared to be opened, and opening it would expose every past commit as well as the current one. Internal material is therefore kept out of the repository altogether rather than merely out of `docs/`: the gaps register lives in `gaps.json`, the DOCX is built from it, and both are in `.gitignore`. Before committing anything new, ask whether it could be read by a stranger - for this repository that is the only test that matters.

## Look and feel

The page follows https://pasteur-network.org, using the tokens read from that site's own stylesheet: Network blue `#00334c` for the masthead band, the footer band and every link, Network orange `#e5721e` for accents, rules and the filter pills, body text in `#17264D`, and light type on the bands in `#A3E7FC`. Headings are set in the Network's stack - Avenir Next, falling back to Helvetica Neue and Arial - and body text in Verdana, exactly as the site sets them. Those live in the `:root` block of `build_web.py`; change them there, not in the generated HTML.

Orange is never used for running text. At roughly 3:1 against white it fails the WCAG AA contrast threshold of 4.5:1 for body copy, which is why links are navy and orange is kept for fills, rules and hover states - the same way the Network's own site uses it.

The logo is `pasteur_network_logo.png` at the repository root. It is a build input, not a published file: `build_web.py` reads it and base64-encodes it into the page, so swapping the artwork means replacing that file and rebuilding, with no code change. The supplied artwork is black on transparent, so the stylesheet inverts it to white for the navy band and turns the inversion off again for print, where the band prints white.

Three things are deliberately *not* taken from the Network site. The domain and access colours - health, climate, environment, reference, derived, and open/by-request/restricted - carry meaning, so they stay as they are rather than collapsing into the single accent. The small uppercase labels stay monospace, because they mark structure rather than voice, and the Network site has no monospace face to borrow. And the headings are smaller than the site's 60px h1, which would overwhelm a dense register.

## Who names what

Three different things name an institution, and they are not interchangeable.

- **The publisher** is the Pasteur Network. It is `catalogue.institution`, and it sets the page title, the DOCX cover and the attribution string.
- **The contributing member** is the institute whose work these records describe - today `catalogue.member`, the Institut Pasteur de Sao Paulo (IPSP). It appears in the masthead line beside the publisher.
- **The steward** is per record. `steward` names who is answerable for *that* record, and the 26 records here name the IPSP Climate and Health Laboratory because that is the factual answer. A record contributed by another member names that member's own team.

The same distinction governs `custodian`, `producer` and `contact`: those describe who holds and who produced the data, so they stay factual whatever the catalogue is branded as. The rule is simple - branding fields follow the publisher, descriptive fields follow the truth.

## Field standard

The 27 fields follow DCAT (the EU and Brazilian government standard for data catalogues) with four additions that matter for climate–health work and are not in DCAT: **analysis unit**, **latency**, **linkage key** and **known limitations**. Latency is

the field that decides whether a dataset can support an operational decision. Linkage key is the field that makes the catalogue usable rather than merely descriptive.

Each record carries two further keys that are **not** catalogue fields and are not in the CSV: `spatial_grain` and `temporal_grain`, whole numbers from 1 to 5 that drive the coarse-to-fine resolution mark on the web page. They are a presentation device, kept beside the record they describe so that a new dataset cannot quietly inherit a default.

Any organisation joining the catalogue fills in these fields for its own datasets and keeps its own data where it is. That is all the coordination required.

## Licence

Two kinds of material, licensed separately in `LICENCE`:

- **The build scripts** - `build_web.py`, `build_docx.js`, `convert2pdf.py`, `convert2docx.py`, and the HTML, CSS and JavaScript they emit - under the **MIT Licence**.
- **The catalogue itself** - `catalogue.json`, the CSV register, the DOCX, the records on the published page, and this guide - under **CC BY 4.0**. Reuse it anywhere, including commercially, provided you attribute:

> Pasteur Network Climate and Health Metadata Catalogue, Pasteur Network, https://pasteurnetwork.github.io/climate-health-data-catalogue/

Copyright (C) 2026 Pasteur Network, Institut Pasteur de Sao Paulo (IPSP) and Mauro Cesar Cafundo de Morais.

The distinction that matters: **this licence covers the descriptions, not the datasets they describe.** The catalogue is metadata. Each record's `licence` field carries the terms of the underlying data, and several of those are held by third parties or governed by a data-use agreement - nothing here grants any right to them.